

SENIOR PROJECT II



DiscoVeR,
<https://humanfromnowhere.github.io/discover.github.io/>

Project Team Members:

Levent BEKTEMUR, Database Manager
Dilge Nisa ÜSTÜNTAŞ, Project Manager
Süzanna NEBİOĞLU, Product Designer

Supervisor:

Ulaş Güleç

Jury Members:

Tolga Kurtuluş Çapın
Tansel Dökeroğlu

Coordinator:

Dr. Özlem ALBAYRAK

Table of Contents

1- INTRODUCTION and SCOPE.....	4
1.1 Object-Design trade-offs.....	4
1.2 Interface documentation guidelines.....	4
1.3 Engineering standards (UML models).....	4
2- PACKAGES.....	6
2.1 WebSunucusu (Web Server Layer).....	6
2.2 c492 (Unity WebGL Application).....	6
2.3 node_modules (External Dependencies).....	6
3- CLASS INTERFACES.....	7
3.1 Component 1: Unity Visualization Engine.....	7
3.2 Component 2: Web Interface and Integration Layer.....	7
3.3 Interface Relationships and Data Flow.....	8
4- GLOSSARY.....	8
5- REFERENCES.....	9

Table of Tables

Table 1: Related Documents	3
Table 2: Change History	4
Table 3: Term definitions and acronyms table.	9

RELATED DOCUMENTS

Name	URL
High Level Design	https://humanfromnowhere.github.io/discover.github.io/

Table 1: Related Documents

CHANGE HISTORY

Date	Changed by	Change Description
13.03.2026	Dilge Nisa Üstüntaş	Introduction, Glossary
14.03.2026	Süzanna Nebioğlu	URLs, tables, packages
14.03.2026	Levent Bektemur	Class Interfaces, References

Table 2: Change History

1- INTRODUCTION and SCOPE

This document serves as the comprehensive Low-Level Design (LLD) specification for the DiscoVeR Virtual Reality Touring Platform. It outlines the software packages, class-level interfaces, and internal architecture needed to successfully develop the system. This report gives the development team the exact blueprints needed to code the Single Page Application (SPA) frontend, the backend services, and the Unity VR environment integration by moving from the High-Level Design to detailed, technical requirements.

1.1 Object-Design trade-offs

A number of architectural trade-offs were assessed during the design stage. We prioritized the separation of concerns and rigorously followed object-oriented programming techniques to guarantee maintainability. Logic controllers (JavaScript) and view components (HTML/CSS) are separated in the React frontend, which increases the number of files but greatly improves code readability and modularity. By using a Single Page Application strategy instead of multi-page routing and integrating the 3D environment directly into the main dashboard view, we were able to trade somewhat higher initial load times for a smooth, continuous user experience for the Unity VR integration.

1.2 Interface documentation guidelines

Standard RESTful principles are used to document all programmatic interfaces, including the Unity-to-React bridge functions and backend API endpoints. To enable auto-generated documentation, data payloads are formatted in JSON, and internal class methods are documented using JSDoc for JavaScript modules and inline standard XML comments for C# scripts.

1.3 Engineering standards (UML models)

The Unified Modeling Language (UML) standards are followed in the system's behavioral and structural designs. Sequence diagrams are used to map out intricate asynchronous workflows, including user authentication and 3D model retrieval, whereas class diagrams are used to specify the properties, methods, and access modifiers of individual components.

Definitions, Acronyms

Term	Acronym	Definition
Application Programming Interface	API	A set of rules and protocols that help two different software applications to communicate and exchange data.
JavaScript Object Notation	JSON	Open standard file format that uses plain text for data storing and transporting.
Single Page Application	SPA	A web application that loads contents dynamically without the need to refresh the whole page.
User Interface / User Experience	UI/UX	User interface is the interactivity, feel and looks of a web page or product. User experience on the other hand covers the user's overall experience with the product.
Virtual Reality	VR	A 3D environment that is generated by a computer that simulates an immersive, realistic experience.
GL Transmission Format Binary	GLB	Binary file format for 3D models based on the glTF standard.
Filmbox	FBX	File format moving 3D animation scene data between software packages.
Wavefront Object	OBJ	"Very widely supported open file format for the representation of 3D geometry." (DANS. n.d.)

Table 3: Term definitions and acronyms table.

2- PACKAGES

The **DiscoVeRGit** system architecture is designed with a clear separation of concerns, divided into three primary packages that handle server-side logic, client-side 3D rendering, and external dependencies.

2.1 WebSunucusu (Web Server Layer)

This package acts as the backbone of the application, responsible for hosting the platform and managing data flow.

- **Core Responsibility:** Operates as the HTTP API server and provides static file hosting for the 3D visualization assets.
- **Logic Encapsulation:** It encapsulates the Express.js server logic (`server.js`), handles CORS (Cross-Origin Resource Sharing) to allow secure communication between the web UI and Unity, and manages the `/uploads` directory for OBJ model storage.
- **Interactions:** It serves the web user interface to the client's browser and acts as a bridge by providing model files to the Unity WebGL client upon request.

2.2 c492 (Unity WebGL Application)

This package contains the core visualization engine developed within the Unity environment.

- **Core Responsibility:** Manages 3D model rendering, mesh generation, and interactive user navigation.
- **Logic Encapsulation:**
 - * **Camera Control:** Implements orbital navigation, zooming, and panning through `Kamera.cs`.
 - **Model Loading:** Contains a custom OBJ parser (`ModelYukleyici.cs`) that handles the conversion of vertex and face data (including quad-to-triangle conversion) into dynamic Unity meshes.
 - **Environmental Logic:** Manages automatic lighting and scene management via the `Yonetici` prefab.
- **Interactions:** It fetches raw OBJ data from the WebSunucusu package and receives state-change commands from the Web UI via the Unity Messaging API.

2.3 node_modules (External Dependencies)

This package manages the third-party integrations required for system stability and extended functionality.

- **Core Responsibility:** Provides the runtime environment for the Node.js server and integrates backend services.
- **Key Dependencies:** Includes `express` for routing, `cors` for security policy management, and `@supabase/supabase-js` for potential future database and authentication integration.

3- CLASS INTERFACES

This section defines the internal structure of the DiscoVeRGit system, detailing the classes, their responsibilities, and the method signatures that enable 3D model visualization and user interaction.

3.1 Component 1: Unity Visualization Engine

This component is responsible for the core 3D rendering and camera physics within the WebGL environment.

3.1.1 Class: Kamera

- **Inheritance:** [MonoBehaviour](#)
- **Responsibility:** Provides orbital camera mechanics, allowing users to rotate, zoom, and pan around the 3D model.
- **Key Methods:**
 - [void Start\(\)](#): Initializes the camera's location and distance relative to the target.
 - [void LateUpdate\(\)](#) (Private): Processes per-frame mouse input to update the camera's spherical coordinates (\$x, y, mesafe\$).

3.1.2 Class: ModelYukleyici

- **Inheritance:** [MonoBehaviour](#)
- **Responsibility:** Manages the lifecycle of 3D assets, from server-side fetching to runtime mesh generation.
- **Key Methods:**
 - [void ModelYukle\(string gelenIsim\)](#): Public interface to produce a new model load. It resets the scene and begins the download process.
 - [IEnumerator IndirVeOlustur\(string url\)](#) (Private): A coroutine that handles the asynchronous web request to fetch OBJ data.
 - [Mesh OzelObjOkuyucu\(string data\)](#) (Private): The heart of the parsing engine that converts raw string data into a Unity [Mesh](#) object, including quad-to-triangle conversion logic.

3.2 Component 2: Web Interface and Integration Layer

This component manages the User Interface (UI) and the bridge between the browser's JavaScript engine and the Unity WebGL instance.

3.2.1 Module: Viewer (viewer.js)

- **Responsibility:** Coordinates the initialization of the Unity instance and handles high-level display commands.
- **Key Functions:**
 - [void ModelAc\(string dosyaAdi\)](#): Activates the Unity container and uses the [SendMessage](#) API to communicate with the [ModelYukleyici](#) class in Unity.

- `void UnityGizle()`: Manages UI state by hiding the 3D viewport when not in use.

3.2.2 Module: UI Controllers (IndexStart / AdminStart)

- **Responsibility:** Manages application state, tab navigation, and user event handling for both standard users and administrators.
- **Key Functions:**
 - `void switchTab(string tabName)`: Toggles visibility between "Dashboard" and "Models" sections.
 - `void showAdminTab(string name)`: Handles administrative view transitions for model uploads and management.

3.3 Interface Relationships and Data Flow

The integration between components follows a specific communication protocol:

1. **Unity-JavaScript Bridge:** Communication is achieved via the `unityErisim.SendMessage('Yonetici', 'ModelYukle', modelName)` call, where the JavaScript layer invokes methods on the Unity `Yonetici` prefab.
2. **Data Flow:**
 - **Trigger:** User clicks a model button in the Web UI.
 - **Command:** `viewer.js` sends the filename to Unity.
 - **Fetch:** `ModelYukleyici.cs` performs an HTTP GET request to the Node.js server (`/uploads`).
 - **Render:** The OBJ parser generates the mesh, and `Kamera.cs` enables user interaction with the newly rendered object.

4- GLOSSARY

- **API (Application Programming Interface):** A collection of guidelines and procedures that enable communication between the frontend and the backend database.
- **CORS (Cross-Origin Resource Sharing):** A web browser security feature that regulates resource sharing and HTTP requests between various domains or origins.
- **Coroutine:** A Unity (C#) structure that allows asynchronous tasks, such as downloading OBJ files over several frames, to be carried out without stopping the main rendering cycle.
- **Express.js:** A simple and adaptable Node.js web application framework for handling server-side logic, routing, and HTTP API calls.
- **JSON (JavaScript Object Notation):** A lightweight data-interchange format used for transmitting data between the server and the web application.
- **Lucide React:** A vector icon library used within the frontend view components to maintain a stable and scalable UI design.
- **Mesh:** A collection of vertices, edges, and faces that defines the 3D shape and structural outline of an object within the Unity rendering engine.
- **MonoBehaviour:** The base class in the Unity game engine from which every Unity script derives, used to manage the lifecycle and events of 3D objects.
- **OBJ (Wavefront .obj):** A standard 3D model format that stores geometric data, including vertex positions and polygon faces, in a plain text format.
- **Quad-to-Triangle Conversion:** The automated algorithmic process of converting quadrilateral polygon faces from an OBJ file into triangular faces, which is necessary for 3D graphics engines like Unity to render the models properly.
- **REST (Representational State Transfer):** An architectural style used for designing the backend networked applications and data-fetching APIs.
- **SPA (Single Page Application):** A web application implementation where the login, dashboard, and VR environments are dynamically rewritten on a single web page, such as index.html, rather than loading entirely new pages from the server, ensuring smoother transitions.
- **Supabase:** A backend-as-a-service platform integrated to provide database management and possible user authentication for the system.
- **Unity Messaging API (SendMessage):** A communication bridge that allows JavaScript code in the web browser, such as [viewer.js](#), to invoke specific methods on compiled Unity WebGL game objects, such as the [Yonetici](#) prefab.
- **VR (Virtual Reality):** A simulated 3D environment that users can explore and interact with, which is the heart of the touring feature of the DiscoVeR platform.
- **WebGL (Web Graphics Library):** A JavaScript API utilized to render the Unity 3D interactive graphics directly within the web browser without requiring external plugins.

5- REFERENCES

DANS. (n.d.). *Wavefront Object (.obj)*. Data Archiving and Networked Services (DANS). <https://dans.knaw.nl/en/file-formats/3d/wavefront-object/>

DiscoVeR Project Team. (2026). *High-Level Design Document*. TED University SENG492.

DiscoVeR Project Team. (2025). *Software Requirements Specification (SRS)*. TED University SENG491.

Lucide Contributors. (n.d.). *Lucide React Icons*. Retrieved from <https://lucide.dev/>

Meta / React Core Team. (n.d.). *React Documentation*. Retrieved from <https://react.dev/>

Unity Technologies. (n.d.). *Unity WebGL Player Scripting Interaction*. Retrieved from <https://docs.unity3d.com/Manual/webgl-interactingwithbrowserscripting.html>